КАК УСТРОЕНО ПРИЛОЖЕНИЕ «МАРШРУТ»

Памятка для подготовки к защите: зачем нужно приложение, что делает каждый экран, где что лежит в коде и как коротко ответить на вопрос преподавателя.

1. Идея в двух словах

«Маршрут» сам составляет личный план обучения. Пользователь выбирает, что хочет изучить (одно из четырёх направлений), проходит короткий тест, указывает, сколько часов в неделю готов заниматься и к какому сроку нужно успеть. Приложение убирает темы, которые человек уже знает, ставит остальные в правильном порядке (сначала основы, потом то, что на них опирается), раскладывает их по дням и добавляет повторения. Если человек пропустил занятия, план перестраивается одной кнопкой.

Всё работает без интернета и без сервера: аккаунты, планы и заметки хранятся в базе SQLite на самом устройстве. Приложение написано на Flutter, поэтому один и тот же код собирается для Android и для Windows.

В каталоге 4 направления: «Flutter-разработчик» (20 тем), «Python с нуля» (18), «Английский A1–B1» (19) и «Алгоритмы и структуры данных» (18). Всего 75 тем, 113 связей «эту тему изучать после той» и 32 вопроса для теста. Каталог лежит в файлах `assets/catalog/*.json`, поэтому новое направление добавляется без изменения кода.

2. Экраны по порядку

Таблица 1 — Экраны приложения в том порядке, в каком их видит пользователь

Экран	Что видит и делает пользователь	Файл
Заставка (Splash)	Логотип «М» и название. Приложение проверяет, отмечал ли пользователь «Запомнить меня»: если да, сразу открывается главная, иначе — вход	`screens/splash_screen.dart`

Экран	Что видит и делает пользователь	Файл
Регистрация	Имя, email, пароль и повтор пароля. Ошибки показываются под полями: пустое поле, неверный email, слабый пароль, пароли не совпадают, email уже занят	`screens/auth/register_screen.dart`
Вход	Email и пароль, флажок «Запомнить меня», переход к регистрации. При неверных данных — сообщение «Неверный email или пароль»	`screens/auth/login_screen.dart`
Онбординг, шаг 1: направление	Выбор одного из четырёх направлений	`screens/onboarding/direction_screen.dart`
Шаг 2: тест	Вопросы по темам направления, вариант «Не знаю». Новичок может пропустить тест кнопкой «Я новичок»	`screens/onboarding/diagnostic_screen.dart`
Шаг 3: расписание	Часы в неделю, учебные дни, дата начала, дедлайн (можно без срока). Кнопка «Сформировать план»	`screens/onboarding/schedule_screen.dart`
Шаг 4: предпросмотр	Сколько тем и часов, какие темы убраны по тесту, дата окончания. Если в дедлайн не успеть — подсказка «нужно N ч в неделю» и кнопка, которая сразу ставит это значение. Кнопка «Начать обучение» сохраняет план	`screens/onboarding/preview_screen.dart`
Главная	Приветствие, кольцо прогресса, серия дней подряд, сколько дней до дедлайна, занятия на сегодня с отметкой. Если есть пропуски — баннер «Пропущено N занятий» с кнопкой «Перестроить». Запуск Pomodoro	`screens/home/home_screen.dart`
План	Все занятия по дням. Фильтры «Сегодня», «Предстоящие», «Выполненные», «Пропущенные», «Повторения», поиск по темам. Меню: настройки плана и «Мои планы»	`screens/plan/plan_screen.dart`
Занятие	Описание темы и материалы, отметка «Выполнено» и её отмена, для повторения — подсказка «Как повторять», запуск таймера	`screens/plan/session_screen.dart`
Настройки плана	Те же поля, что в шаге 3. «Сохранить и перестроить» пересчитывает план от сегодняшнего дня	`screens/plan/plan_settings_screen.dart`
Мои планы	Несколько планов: сделать активным, переименовать, перестроить, удалить, создать новый	`screens/plan/plans_screen.dart`
Темы	Каталог направления: поиск по названию и описанию, фильтр по категориям, вкладки «Избранное» (звёздочка) и «Мои» (свои темы)	`screens/topics/topics_screen.dart`
Тема и своя тема	Описание, материалы, от каких тем зависит, заметки к теме. Свою тему можно создать, изменить, удалить и добавить в активный план — план перестроится	`screens/topics/topic_screen.dart`, `custom_topic_screen.dart`

Экран	Что видит и делает пользователь	Файл
Статистика	Часы учёбы, число занятий, число «помидоров». График часов по неделям, круговая диаграмма по категориям, история выполненных занятий	`screens/stats/stats_screen.dart`
Профиль	Аватар, имя, email, дата регистрации, итоги. Редактирование профиля, смена пароля, тема оформления (светлая, авто, тёмная), ежедневное напоминание, «Мои заметки», «Мои планы», выход, удаление аккаунта	`screens/profile/`
Заметки	Список заметок с поиском, создание, изменение и удаление, привязка к теме	`screens/notes/`
Pomodoro	Таймер 25/5 или 50/10 минут: старт, пауза, сброс. Когда фокус закончился — уведомление, запись в статистику и перерыв	`screens/pomodoro/pomodoro_screen.dart`

Пути в таблице даны от папки `lib/presentation/`. Нижняя панель с пятью вкладками (Главная, План, Темы, Статистика, Профиль) описана в `screens/shell/main_shell.dart`; в широком окне Windows она превращается в боковую панель.

3. Как всё устроено внутри

3.1 Три слоя

Код разделён на три слоя по принципам чистой архитектуры. Зависимости направлены внутрь: интерфейс и данные знают о предметной области, а предметная область ни о ком не знает.

- **`lib/domain` — предметная область.** Чистый Dart без Flutter и базы данных: сущности (тема, план, занятие, настройки расписания), алгоритм построения плана (`planner/`), интерфейсы репозиториев (`repositories/repositories.dart`) и сценарий работы с планом `usecases/plan_service.dart`. Поэтому алгоритм проверяется быстрыми модульными тестами.
- **`lib/data` — данные.** База SQLite через пакет drift (`db/app_database.dart`), реализации репозиториев (`repositories/`), чтение каталога из JSON (`datasources/`), уведомления (`services/reminder_service_impl.dart`).

- **`lib/presentation` — интерфейс.** Экраны (`screens/`), общие виджеты (`widgets/`) и провайдеры Riverpod (`providers/`), которые связывают экраны с данными.
- **`lib/core` — общее:** маршруты и правило доступа (`router/app_router.dart`), тема оформления (`theme/app_theme.dart`), хэширование паролей и проверка полей (`utils/`).

Состоянием управляет **Riverpod**: экран не создаёт объекты сам, а берёт их у провайдеров (`providers/providers.dart`). В тестах провайдеры подменяются: база в памяти, фиксированная дата, быстрое хэширование.

3.2 Как строится план

Алгоритм лежит в `lib/domain/planner/` и работает в пять шагов (`plan_generator.dart`):

1. **Граф тем.** Темы направления — это вершины, а связь «В изучать после А» — ребро. Получается ориентированный граф без циклов.
2. **Исключение известных тем** (`knowledge_estimator.dart`). Верный ответ в тесте означает, что известны сама тема и все темы, на которых она стоит. Неверный ответ или «Не знаю» означает, что тема не известна, даже если её «покрыл» верный ответ по более сложной теме.
3. **Порядок тем** — топологическая сортировка, алгоритм Кана (`topological_sort.dart`). Берутся темы, у которых не осталось неизученных предшественников; среди них первой идёт более простая. Если в данных цикл, алгоритм это обнаруживает и сообщает об ошибке.
4. **Раскладка по дням** (`scheduler.dart`). Дневной лимит — часы в неделю, делённые на число учебных дней (округляется до 15 минут, не меньше 30). День за днём: сначала повторения, срок которых наступил (по 15 минут), потом текущая тема; большая тема делится на части не

короче 30 минут. После каждой изученной темы ставятся повторения через 1, 3 и 7 дней — по кривой забывания Эббингауза.

5. **Проверка дедлайна.** Если последнее занятие выходит за срок, приложение подбирает с шагом 0,5 часа минимальное число часов в неделю, при котором план успевает (не больше 84 часов, это 12 часов в день).

Пример из курсовой (таблица 2.4): три темы по 60 минут, «Основы Dart» → «Виджеты» → «Состояние», 7 часов в неделю, все дни, дедлайн через три дня. Дневной лимит — 60 минут. 1-й день: «Основы Dart» 60 минут. 2-й день: повторение «Основ Dart» 15 минут и первая часть «Виджетов» 45 минут. 3-й день: вторая часть «Виджетов» 15 минут и первая часть «Состояния» 45 минут. Последняя часть «Состояния» попадает на 4-й день, то есть позже дедлайна, поэтому приложение предлагает 8 часов в неделю: лимит становится 75 минут, и план укладывается в срок.

3.3 Перепланирование

`replanner.dart` вызывается кнопкой «Перестроить» или после изменения расписания. Выполненные занятия остаются на своих местах. Для каждой темы считается, сколько минут уже изучено, остаток заново раскладывается с сегодняшнего дня теми же правилами, а пропущенные повторения переносятся на ближайшие учебные дни. Номера частей тем продолжаются: если изучены части 1 и 2, следующей будет часть 3.

3.4 База данных

8 таблиц SQLite в `lib/data/db/app_database.dart`: пользователи (`users`), планы (`plans`), темы плана в порядке изучения (`plan_topics`), занятия (`sessions`), свои темы (`custom_topics`), заметки (`notes`), избранное (`favorites`) и записи Pomodoro (`pomodoro_logs`). Все связи объявлены внешними ключами с каскадным удалением: удалили план — удалились его

занятия, удалили аккаунт — удалились все данные пользователя. Сам каталог тем в базе не хранится, он читается из JSON.

Файл базы лежит в служебной папке приложения (в Windows — `%APPDATA%`). Кто вошёл в систему и какая тема оформления выбрана, хранится отдельно, в `shared_preferences` (`session_storage_impl.dart`).

3.5 Регистрация, вход и пароли

`lib/data/repositories/auth_repository_impl.dart` и `lib/core/utils/password_hasher.dart`. Пароль не хранится: в базу записываются случайная соль (16 байт) и хэш PBKDF2-HMAC-SHA256 со 100 000 повторений. Расчёт идёт в отдельном изоляте, чтобы интерфейс не замирал, а хэши сравниваются за постоянное время. В первом семестре был SHA-256 с солью; такие хэши пересчитываются в новый формат при следующем входе.

«Запомнить меня» сохраняет id пользователя в `shared_preferences`, и при следующем запуске заставка сразу открывает главную. Состояние входа держит `providers/auth_controller.dart`.

3.6 Навигация и защита экранов

Маршруты описаны в `lib/core/router/app_router.dart` (пакет go_router). Перед каждым переходом срабатывает одна функция `authRedirect`: пока сессия проверяется, виден только Splash; без входа доступны только вход и регистрация; после входа экраны входа ведут на главную. Поэтому после выхода или удаления аккаунта закрытые экраны сразу становятся недоступны. Каждая вкладка нижней панели помнит свою прокрутку и фильтры (`StatefulShellRoute`).

3.7 Уведомления

`lib/data/services/reminder_service_impl.dart`, пакет flutter_local_notifications. На Android работают ежедневное напоминание в

выбранное время (включается в профиле) и мгновенное уведомление об окончании «помидора». В Windows плагин не умеет повторять уведомление каждый день без установщика MSIX, поэтому там переключатель напоминания неактивен с подписью «Доступно в версии для Android и iOS», а уведомление Pomodoro работает.

3.8 Pomodoro

`lib/presentation/screens/pomodoro/pomodoro_screen.dart`. Два режима: 25 минут фокуса и 5 перерыва или 50 и 10. Оставшееся время считается от момента окончания интервала, а не уменьшением счётчика, поэтому таймер не отстаёт. Законченный интервал записывается в таблицу `pomodoro_logs` и попадает в статистику.

3.9 Тесты и сборка

97 модульных и виджет-тестов в папке `test/` (алгоритм, база, экраны, правило доступа, адаптивность) и интеграционный тест настоящего приложения в `integration_test/`: регистрация → вход → создание плана → профиль → выход. При каждой отправке кода GitHub Actions (`.github/workflows/build.yml`) запускает анализ и все тесты, затем собирает релизный APK для Android и программу для Windows. Матрица из 280 снимков всех экранов (`tool/screenshots/qa_matrix_test.dart`) помогла найти ошибки вёрстки; всего найдено и исправлено 24 ошибки (`docs/coursework/testing.md`).

4. Если спросят

Таблица 2 — Короткие ответы на вероятные вопросы

Вопрос	**Ответ**
Зачем это приложение?	Чтобы человек, который учится сам, получил готовый план под своё время и срок, а не составлял его вручную
Чем отличается от roadmap.sh, Duolingo, Coursera?	Они дают схему тем, мотивацию или курсы, но ни одно не учитывает знания пользователя, его часы и дедлайн одновременно и не перестраивает план при отставании

Вопрос	Ответ
Почему Flutter?	Один код для Android и Windows, собственная отрисовка (везде выглядит одинаково), горячая перезагрузка, Material 3 из коробки
Что такое чистая архитектура в проекте?	Три слоя: domain (алгоритм и сущности, чистый Dart), data (база и репозитории), presentation (экраны). Алгоритм не зависит от Flutter и базы, поэтому легко тестируется
Почему Riverpod?	Проверка зависимостей при компиляции, нет зависимости от дерева виджетов, легко подменять данные в тестах
Почему SQLite (drift), а не Firebase?	Приложение должно работать без интернета и без ключей; связи между таблицами и каскадное удаление есть в SQLite из коробки; drift генерирует типобезопасный код
Как строится порядок тем?	Топологической сортировкой графа зависимостей (алгоритм Кана), сложность O((V + E) · log V)
Что будет, если в каталоге ошибка — цикл зависимостей?	Алгоритм Кана это обнаружит: останутся темы, которые нельзя поставить, и приложение покажет понятное сообщение
Как работает тест знаний?	Верный ответ отмечает тему и все её основы как известные, неверный ответ или «Не знаю» оставляет тему в плане
Откуда интервалы повторений 1, 3, 7 дней?	Из кривой забывания Эббингауза и эффекта распределённого повторения: повторять лучше с растущими промежутками
Что если не успеваю к дедлайну?	Приложение подбирает минимальное число часов в неделю с шагом 0,5 и предлагает установить его одной кнопкой
Что если пропустил занятия?	Кнопка «Перестроить»: выполненное остаётся, остаток раскладывается с сегодняшнего дня
Как хранятся пароли?	Только хэш PBKDF2-HMAC-SHA256 со случайной солью и 100 000 повторений; сам пароль нигде не сохраняется
Как закрыты экраны без входа?	Одной функцией redirect в роутере: без входа открываются только вход и регистрация
Где реализованы CRUD-операции?	Заметки, свои темы и планы: создание, просмотр, изменение, удаление, данные в SQLite, ошибки показываются понятным сообщением
Как проверяли приложение?	97 автоматических тестов, интеграционный тест, 36 тест-кейсов, ручные проверки и 280 снимков экранов; найдено и исправлено 24 ошибки
Как установить приложение?	APK для Android и архив для Windows собирает GitHub Actions; их можно скачать со страницы сборки или релиза